Falsifiable Assurance in Agentic AI Systems

via Append-Only Cryptographic Audit Chains

Ahmad Meta[1]

[1]SnapKitty Sovereign AI Research , github.com/SNAPKITTYWEST

June 2026 arXiv:cs.AI / cs.CR / cs.SE

Abstract

Contemporary AI governance frameworks rely on *claimed* behavioral guarantees—audit logs that are mutable, access controls that can be overridden, and compliance reports generated after the fact. We propose a fundamentally different model: **falsifiable assurance**, in which every AI decision, tool invocation, and model output is immediately sealed into a cryptographically chained, append-only ledger that cannot be altered without detection.

We introduce the **WORM Audit Chain** (Write Once Read Many), a SHA-256 linked ledger applied at the boundary of each agent action in a multi-step agentic workflow. Combined with the **Trust Deed**—a declarative governance contract evaluated before every mutation—the system produces behavioral guarantees that are *externally verifiable* and *falsifiable by construction*.

We demonstrate the architecture on the **Frankenstein Workflow**, a four-stage agentic pipeline (Brain → Hands → Legs → Review) processing 1,247 real enterprise tasks across ERP, code review, and CI/CD domains. We measure *Assurance Density* (sealed decisions per workflow step), *Chain Integrity Rate* (proportion of seals surviving tamper detection), and *Trust Deed Rejection Rate* (governance violations caught pre-execution).

Our results show that falsifiable assurance adds a median 4 ms overhead per agent action while providing a complete, tamper-evident behavioral record that satisfies SOX §302, GDPR Article 5(2), and ISO 27001 Annex A.12.4 audit requirements *without any post-hoc reporting step*. The full implementation is released at `https://github.com/SNAPKITTYWEST`.

Keywords: agentic AI, audit chains, cryptographic integrity, falsifiable assurance, enterprise compliance, WORM ledger, governance

Contents

1 Introduction

1.1 The Auditability Gap

Large language model deployments in enterprise settings face a fundamental tension: the same flexibility that makes LLMs valuable—open-ended reasoning, tool use, multi-step planning—makes them difficult to audit. When a model decides to approve a purchase order, modify a financial record, or trigger a downstream system, that decision is typically logged to a mutable database that an administrator can alter, a log file that can be rotated, or not logged at all.

We term the distance between what a system *claims* it did and what can be *independently verified* the **auditability gap**.

Existing approaches to AI auditability fall into three categories:

1. **Post-hoc explanation** (LIME [Ribeiro et al., 2016], SHAP [Lundberg and Lee, 2017], attention visualization [Jain and Wallace, 2019])—explains model internals but not behavioral records.
2. **Structured logging**—mutable; requires trust in the logging infrastructure itself.
3. **Constitutional AI / RLHF** [Bai et al., 2022, Ouyang et al., 2022]—shapes model behavior but produces no verifiable per-action record.

None of these approaches produce what compliance frameworks actually require: a tamper-evident record of *every decision made, when it was made, and what its inputs were.*

1.2 Contributions

This paper makes four contributions:

1. **The WORM Audit Chain**: a SHA-256 linked, append-only ledger architecture for AI agent actions, with a formal tamper-detection proof (§3).
2. **The Trust Deed**: a declarative pre-execution governance contract that blocks policy-violating actions before they execute (§4).
3. **Falsifiable Assurance**: a formal definition of verifiable AI behavioral guarantees with measurable metrics (§5).
4. **Empirical validation** on 1,247 real enterprise tasks, with mapping to SOX, GDPR, and ISO 27001 (§6).

2 Background

2.1 Agentic AI Workflows

Multi-step agentic pipelines [Yao et al., 2023, Schick et al., 2023, Chase, 2022] decompose complex tasks into sequences of reasoning steps, tool invocations, and state mutations. Each step

is a potential audit point. Existing frameworks treat these steps as ephemeral—computed, outputs passed forward, intermediate state discarded.

Benchmarks such as AgentBench [Liu et al., 2023] and ToolBench [Qin et al., 2023] evaluate agent *capability* but not agent *accountability*.

2.2 Cryptographic Audit Chains

Append-only, hash-linked chains provide tamper-evidence: modifying any entry invalidates all subsequent hashes, making tampering detectable by any party holding the chain [Nakamoto, 2008, Laurie et al., 2014].

We adapt this principle to per-action AI audit, eliminating the need for distributed consensus. The chain is local, deterministic, and verifiable by a third party given only the ledger file.

2.3 Constitutional AI and Governance

Constitutional AI [Bai et al., 2022] and Sparrow [Glaese et al., 2022] define behavioral constraints at training time. We argue these are necessary but insufficient: they shape priors but produce no per-execution record. A well-aligned model that produces no verifiable record is unauditable; falsifiable assurance provides the missing verification layer.

3 The WORM Audit Chain

3.1 Formal Definition

Let $\mathcal{A} = (a_1, a_2, \ldots, a_n)$ be a sequence of agent actions.

Definition 1 (WORM Seal)**.** *A* WORM seal *for action a_i is defined as:*

$$S_i = \mathrm{SHA256}(S_{i-1} \,\|\, t_i \,\|\, \mathrm{serialize}(a_i))$$

where $S_0 = 0^{256}$ is the genesis seal, t_i is a Unix millisecond timestamp, $\|$ denotes concatenation, and serialize *is a deterministic encoding of the action payload.*

The chain $\mathcal{C} = (S_1, S_2, \ldots, S_n)$ is tamper-evident:

Theorem 1 (Tamper Detection)**.** *For any modified chain $\mathcal{C}' = (S_1, \ldots, S_{k-1}, S'_k, \ldots, S'_n)$ where $S'_k \neq S_k$, a verifier holding $\mathcal{C}$ and the original action payloads can detect the modification at position k in $O(n)$ time.*

Proof. By induction. S_k depends on S_{k-1} and a_k via SHA-256. Any modification to a_k produces $S'_k \neq S_k$ with overwhelming probability under the collision-resistance assumption of SHA-256 (preimage resistance: 2^{-256} per query under the random oracle model). Since S_{k+1} is computed from S_k, we have $S'_{k+1} \neq S_{k+1}$, and by induction $S'_j \neq S_j$ for all $j \geq k$. A verifier replays the chain in $O(n)$ steps and detects the first mismatch at position k. □ □

Corollary 1. *Prepending or reordering entries is also detectable, since S_i encodes S_{i-1}: any change to the sequence order invalidates all subsequent seals.*

3.2 Implementation

Algorithm 1 presents the seal procedure.

```python
def worm_seal(action, chain):
    entry = {
        "ts":        unix_ms(),
        "prev_hash": chain.last_hash,
        "payload":   serialize(action),
    }
    entry["this_hash"] = sha256(
        (entry["prev_hash"] + str(entry["ts"]) + entry["payload"]).
            encode()
    ).hexdigest()
    chain.append(entry)          # append-only: no update/delete
    chain.last_hash = entry["this_hash"]
    return entry
```

Listing 1: WORM seal procedure

In our PostgreSQL 16 implementation, tamper resistance is enforced at the schema level:

```sql
CREATE TABLE worm_chain (
  id         BIGSERIAL PRIMARY KEY,
  ts         TIMESTAMPTZ NOT NULL DEFAULT now(),
  prev_hash  CHAR(64)    NOT NULL,
  payload    JSONB       NOT NULL,
  this_hash  CHAR(64) GENERATED ALWAYS AS (
    encode(sha256((prev_hash || payload::text)::bytea), 'hex')
  ) STORED
);

REVOKE UPDATE, DELETE ON worm_chain FROM PUBLIC;
REVOKE UPDATE, DELETE ON worm_chain FROM sovereign_app;
```

Listing 2: PostgreSQL WORM chain schema

The `GENERATED ALWAYS AS` column makes the hash server-computed and non-writable. The `REVOKE` statements eliminate the update/delete attack surface at the database permission level, requiring privilege escalation for any tampering attempt.

3.3 Cross-System Chaining

In multi-service pipelines (e.g., Node.js connector → Elixir IDE → Rust orchestrator), each service maintains its own chain. Cross-system integrity is preserved by including the upstream service's latest seal hash in the first payload of the downstream chain:

$$\texttt{node\_seal}_{42} \rightarrow \texttt{abzu\_seal}_{01}\ (\text{prev} = \texttt{node\_seal}_{42}) \rightarrow \texttt{rust\_seal}_{07}\ (\text{prev} = \texttt{abzu\_seal}_{01})$$

This creates a directed acyclic graph of evidence spanning the entire pipeline, verifiable by replay.

4 The Trust Deed

4.1 Pre-Execution Governance

The Trust Deed is a declarative policy evaluated *before* each state mutation. Unlike post-hoc filtering, pre-execution governance prevents policy violations from entering the WORM chain, maintaining the chain as a record of *permitted actions only*.

```
def trust_deed_check(action, rules):
    for rule in rules:
        if rule.condition(action):
            if rule.effect == BLOCK:
                worm_seal({type: "BLOCKED", action, rule: rule.id})
                raise GovernanceViolation(rule)
            elif rule.effect == WARN:
                worm_seal({type: "WARNED", action, rule: rule.id})
    return PERMITTED
```

Listing 3: Trust Deed evaluation

4.2 Rule Taxonomy

We identify five categories of governance rules observed in enterprise AI deployments (Table 1).

Table 1: Trust Deed rule taxonomy

Category	Example	Effect	Severity
Destructive Ops	`rm -rf /`, `DROP TABLE`	BLOCK	CRITICAL
Scope Escalation	Write outside authorized namespace	BLOCK	HIGH
Financial Integrity	Double-entry imbalance in journal entries	BLOCK	HIGH
Elevated Privilege	Production DB write from dev agent	WARN+CONFIRM	MEDIUM
External Exfiltration	Unauthorized outbound API call	BLOCK	HIGH

4.3 Double-Entry Enforcement

For financial AI pipelines, accounting integrity is enforced as a Trust Deed rule at the database level:

```
CREATE CONSTRAINT TRIGGER enforce_double_entry
  AFTER INSERT ON je_line
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION check_balance();
-- Rejects any commit where sum(debit) != sum(credit)
-- per journal_entry_id; rejection is WORM-sealed.
```

Listing 4: Double-entry constraint trigger

5 Falsifiable Assurance: Definition and Metrics

Definition 2 (Falsifiable Assurance)**.** *A behavioral guarantee G about an agentic system $\mathcal{S}$ is* falsifiable *if and only if there exists a verification procedure V such that, given the WORM chain $\mathcal{C}$ produced by $\mathcal{S}$, $V(\mathcal{C})$ returns* TRUE *if G holds and* FALSE *with a specific counterexample if G is violated.*

This contrasts with *claimed assurance* (policy documents, training objectives) which cannot be verified against a specific execution trace.

5.1 Metrics

Assurance Density (AD).

$$\text{AD} = \frac{|\mathcal{C}|}{|\mathcal{A}|}$$

The ratio of WORM seals to agent actions. AD = 1.0 indicates every action is sealed. In our implementation, AD = 1.0 by construction.

Chain Integrity Rate (CIR).

$$\text{CIR} = 1 - \frac{\text{tamper detections}}{\text{verification runs}}$$

Measured by periodic chain replay. In 90 days of production operation, CIR = 1.0.

Trust Deed Rejection Rate (TDRR).

$$\text{TDRR} = \frac{\text{blocked actions}}{\text{total attempted actions}}$$

Measures governance effectiveness. In our ERP deployment, TDRR = 0.023.

6 Experimental Evaluation

6.1 The Frankenstein Workflow

We evaluate on the Frankenstein Workflow, a four-stage agentic pipeline:

- **Brain**: Claude Sonnet 4.6 (AWS Bedrock)—structured architectural reasoning and task decomposition.
- **Hands**: FAISS + Neo4j + NetworkX—semantic retrieval from a sovereign corpus of 717K lines across 20+ languages.
- **Legs**: Granite Code 3B [Mishra et al., 2024] running on a local NVIDIA RTX 3080—code generation and execution.

- **Review**: Claude Sonnet 4.6 (AWS Bedrock)—output verification against the Brain specification.

Each stage produces one or more WORM seals. The pipeline exposes a REST API and receives events from a GitLab CI/CD webhook integration.

6.2 Dataset

We evaluate on 1,247 real enterprise tasks across three domains:

- **ERP operations** (427 tasks): journal entries, purchase order matching, accounts receivable aging, inventory movements.
- **Code review** (512 tasks): push event analysis, merge request reviews, CI pipeline failure diagnosis.
- **Direct commands** (308 tasks): developer-issued `/snapkitty` commands in GitLab merge request comments.

6.3 Results

Table 2 presents quantitative results.

Table 2: Evaluation results on 1,247 enterprise tasks

Metric	**Value**
Total WORM seals generated	9,847
Assurance Density (AD)	1.00
Chain Integrity Rate (CIR)	1.00
Trust Deed Rejection Rate (TDRR)	0.023
Median seal latency	4 ms
p99 seal latency	11 ms
Brain median latency	14,200 ms
Legs median latency	1,077 ms
End-to-end median latency	27,800 ms
Double-entry violations caught (ERP)	31
Destructive command attempts blocked	7

The 4 ms median seal latency represents a 0.014% overhead on the 27.8 s end-to-end workflow—negligible in practice.

6.4 Compliance Mapping

Table 3 maps WORM chain properties to enterprise regulatory requirements.

Table 3: Regulatory compliance mapping

Regulation	Requirement	Satisfied By
SOX §302	Officer certification of financial controls	WORM chain provides independent verification basis
SOX §404	Management assessment of internal controls	Trust Deed + TDRR = falsifiable control evidence
GDPR Art. 5(2)	Accountability: demonstrate compliance	Chain replay produces complete processing record
GDPR Art. 22	Automated decision-making documentation	Every AI decision sealed with inputs and outputs
ISO 27001 A.12.4	Event logging integrity	SHA-256 chained, append-only, cryptographically sealed

7 Discussion

7.1 Limitations

The WORM chain provides tamper-evidence, not tamper-prevention at the hardware level. A sufficiently privileged attacker (database administrator, system administrator) could delete the chain file. We mitigate this through:

- Off-site chain replication (S3-compatible object storage, sealed per sync)
- Cross-system hash anchoring (upstream seal embedded in downstream chain)
- Periodic third-party verification via the public verification API

The Trust Deed enforces declared policies but cannot catch undeclared threat models. Novel attack patterns not anticipated in the governance rule set will not be blocked.

7.2 Relation to AI Safety

Falsifiable assurance is a *complement* to, not a replacement for, alignment research. Constitutional AI and RLHF shape model priors; falsifiable assurance creates a post-hoc verification layer. Both are necessary: a well-aligned model that produces no verifiable record is unauditable; a fully audited misaligned model is transparent but still harmful.

7.3 Enterprise Deployment

The 4 ms median seal latency represents negligible overhead on workflows with 14–28 second end-to-end latency. We observe no throughput degradation at 50 concurrent workflow requests.

8 Conclusion

We presented *falsifiable assurance*—a formal framework for producing cryptographically verifiable behavioral records from agentic AI systems. The WORM Audit Chain and Trust Deed together provide:

1. A tamper-evident record of every AI decision (Theorem 1)
2. Pre-execution governance that prevents policy violations
3. Measurable, auditable metrics: AD, CIR, TDRR
4. Direct mapping to SOX, GDPR, and ISO 27001 requirements

The architecture is language-agnostic (implemented in Rust, Node.js, Elixir, and PostgreSQL), adds 4 ms median latency, and requires no changes to the underlying model or training procedure.

We release the full implementation under the Sovereign Source License at https://github.com/SNAPKITTYWEST.

References

Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

H. Chase. LangChain, 2022. URL https://github.com/langchain-ai/langchain.

A. Glaese et al. Improving alignment of dialogue agents via targeted human judgements. *arXiv preprint arXiv:2209.14375*, 2022.

S. Jain and B. C. Wallace. Attention is not explanation. In *Proceedings of NAACL-HLT*, 2019.

B. Laurie, A. Langley, and E. Kasper. Certificate transparency. Technical Report RFC 6962, Internet Engineering Task Force, 2014. URL https://datatracker.ietf.org/doc/html/rfc6962.

X. Liu et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

M. Mishra et al. Granite code models: A family of open foundation models for code intelligence. *arXiv preprint arXiv:2405.04324*, 2024.

S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Decentralized business review*, 2008.

L. Ouyang et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

Y. Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

M. T. Ribeiro, S. Singh, and C. Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD*, 2016.

T. Schick et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A WORM Chain Verification Algorithm

```python
import json, hashlib

def verify_chain(chain_file):
    entries = [json.loads(l) for l in open(chain_file)]
    prev = '0' * 64
    for i, entry in enumerate(entries):
        src = (prev + str(entry['ts']) + entry['payload']).encode()
        expected = hashlib.sha256(src).hexdigest()
        if entry['this_hash'] != expected:
            return False, f"Tamper detected at entry {i}: " \
                          f"expected {expected[:16]}..., " \
                          f"got {entry['this_hash'][:16]}..."
        prev = entry['this_hash']
    return True, f"Chain intact: {len(entries)} entries verified"
```

Listing 5: Chain verification procedure

B Trust Deed Rule DSL

```
{
  "version": "1.0",
  "rules": [
    {
      "id":          "no-worm-delete",
      "description": "Prevent deletion of WORM chain entries",
      "pattern":     "TRUNCATE worm_chain|DELETE FROM worm_chain",
      "effect":      "BLOCK",
      "severity":    "CRITICAL"
    },
    {
      "id":          "double-entry",
      "description": "All journal entries must balance",
      "check":       "sum(debit) == sum(credit) per journal_entry_id",
      "effect":      "BLOCK",
      "severity":    "HIGH"
    },
    {
      "id":          "no-root-delete",
      "description": "No recursive deletion of root filesystem",
      "pattern":     "rm -rf /|format c:|del /s \\*",
      "effect":      "BLOCK",
      "severity":    "CRITICAL"
    }
  ]
}
```

Listing 6: Trust Deed governance rule format (JSON)

C Multi-Service Chain Graph

In a three-service deployment, the cross-system WORM graph takes the following structure:

```
# Service A (Node.js)     seal 42
{prev: "0"*64, ts: 1719532800000, payload: {...}, this_hash: "a1b2c3
    ..."}

# Service B (Elixir ABZU IDE)     seals anchor to A's last hash
{prev: "a1b2c3...", ts: 1719532801000, payload: {...}, this_hash: "
    d4e5f6..."}

# Service C (Rust orchestrator)     anchors to B
{prev: "d4e5f6...", ts: 1719532802000, payload: {...}, this_hash: "
    g7h8i9..."}

# Verifier replays A -> B -> C in sequence.
# Any gap in prev_hash linkage indicates a missing or tampered record.
```

Listing 7: Cross-system chain anchoring